Gate Access System — API Reference

Generated from source (lambdas/, template.yaml, shared/python/)

2026-08-19

Contents

Overview **1**

Authentication . . . 1
Authorization model . . . 2
Conventions used in this document . . . 2
User API . . . 3
Account API . . . 8
Estate API . . . 13
Gate API . . . 14
Access Group API . . . 20
Account Type API . . . 22
Privilege API . . . 23
Guest API . . . 24
Vehicle API . . . 27
Event API . . . 28
Event Register API . . . 29
Face API . . . 30
Device Token API . . . 32
Log API . . . 33
Account Type Privilege Link API . . . 34
Gate Access Group Link API . . . 36

Overview

This document is a complete reference for the Gate Access System's HTTP API. The backend is an AWS SAM application: each top-level resource (user, account, estate, gate, guest, vehicle, event, …) is implemented as its own Lambda function, with routes declared in `template.yaml` and dispatched inside each function's `app.py` by matching `httpMethod` + `path` and calling into a handler in `handlers/`.

Every request body is JSON (`event["body"]`, parsed with `json.loads`). Every response is a Lambda-proxy object (`statusCode` + JSON `body`) with permissive CORS headers (`Access-Control-Allow-Origin: *`) attached via a shared `cors_wrap` helper.

Authentication

Most endpoints require a bearer token:

```
Authorization: Bearer <jwt>
```

The token is decoded with `verify_token` (`shared/python/middleware.py`), HS256-signed with a shared `JWT_SECRET`. Depending on how the token was issued, its payload takes one of these shapes:

- **User session token** (from `/user/login`): `{ "user_id": ..., "is_admin": ..., "exp": ... }`

- **Gate controller device token** (from `/device_token/generate_controller_token`): `{ "gate_id": ..., "type": "GATE" }`
- **Gate tablet device token** (from `/device_token/generate_tablet_token`): `{ "gate_id": ..., "type": "TABLET" }`
- **Short-lived action tokens** (OTP/registration/password-reset flows): carry an `email` or `phone_number` claim and are single-purpose, issued by endpoints like `/user/send_otp` or `/user/forgot_password`.

A handful of endpoints (OTP send/verify, registration, login, Google OAuth, some read-only system dashboards) require no token at all — these are called out explicitly in each endpoint's **Auth** line.

Authorization model

Two layers of authorization sit on top of "is this a valid token":

1. **Global admin** — `users.is_admin = true`, embedded in the JWT as `is_admin`. A global admin passes *every* privilege check in the system, regardless of estate. Created via `POST /user/register-admin` (bootstrap, passkey-gated) or `POST /user/promote-admin` (promote an existing user, admin-gated).
2. **Estate-scoped privileges** — an estate admin is not a boolean flag but a `personal_info` row whose `account_type_id` points at the `ADMIN` account type (created via `POST /account/admin-signup`). Authorization is resolved through `account_types → account_type_privilege_links → privileges`, exposed to handler code via the `PrivilegeContext` helper (`shared/python/utils.py`) and its `get_user_privileges_for_*` constructors. Common checks include `can_manage_all()`, `can_manage_accounts()`, `can_manage_gates()`, `can_manage_access_groups()`, `can_manage_devices()`, `can_manage_guests()` / `can_create_guests()`, `can_manage_vehicle_for_person()`, `can_manage_events()`, and `can_view_logs()`. Each is effectively `is_admin OR MANAGE_ALL OR <specific privilege>`, scoped to the relevant estate. Some modules (`log`, part of `event`/`event_register`) still use an equivalent raw SQL check rather than `PrivilegeContext`, per the in-repo `PRIVILEGE_SYSTEM_MIGRATION_GUIDE.md`.

Conventions used in this document

- **Auth** — what the endpoint requires: no token, any valid token, a specific device-token type, `is_admin=true`, or a named privilege.
- **Request body / params** — fields read from the JSON body, path, or query string, and whether each is required.
- **Success response** — status code and JSON shape.
- **Error responses** — status codes and the conditions that trigger them.
- Notable implementation quirks or bugs discovered while reading the handler code are called out inline rather than omitted, so this document reflects actual behavior rather than intended behavior.

This reference covers three Lambda-backed API modules of the gate-access system: **User**, **Account**, and **Estate**. All three lambdas share a common request pattern: `event["body"]` is parsed as JSON, routed by `path` and `httpMethod` in each lambda's `app.py`, and dispatched to a handler function. Authenticated endpoints expect a `Bearer <JWT>` token in the `Authorization` header; the token is decoded with `verify_token` (in `shared/python/middleware.py`) and yields a payload containing at least `user_id` and `is_admin`. Responses are JSON Lambda-proxy objects (`statusCode` + JSON `body`) with CORS headers attached via `cors_wrap`.

Privilege checks beyond simple `is_admin` use the `PrivilegeContext` helper (`shared/python/utils.py`), obtained via functions such as `get_user_privileges_for_estate`, `get_user_privileges_for_account_holder`, `get_user_privileges_for_personal_info`, or `get_user_privileges`. These resolve the caller's account holder in the relevant estate and expose checks like `can_manage_accounts()`, `can_manage_all()`, and `can_manage_sub_users()`. A global admin (`is_admin=true` on the JWT) automatically passes every privilege check.

User API

Base path `/user/...`, implemented in `lambdas/user/` (`app.py` dispatches to `handlers/auth.py` for registration/login/promotion and `handlers/user.py` for account management, password reset, and reports). Most mutating and listing endpoints require a valid Bearer token; several admin actions additionally require `is_admin=true` on the token payload.

POST /user/send_otp

Auth: none.

Request body: `email` (required) — address to send a registration OTP to.

Success response: `200` — `{ "message": "...", "token": "<otp-token>", "email result": <send_email result> }`. The token embeds the email and is required to complete `verify_otp`.

Error responses: `400` if `email` missing.

Generates a 6-digit OTP, stores it in `pending_users` (10-minute expiry), and emails it to the address.

POST /user/verify_otp

Auth: none (uses the OTP token issued by `send_otp` or `register`).

Request body: `token` (required, from send_otp/register), `otp` (required) — the OTP the user received.

Success response: `201` — `{ "message": "Account verified and created successfully.", "user_id": <id> }`.

Error responses: `400` missing token/OTP or invalid token; `404` no pending registration found; `401` invalid or expired OTP.

Verifies the OTP against `pending_users`, creates the row in `users` (copying the pending password/phone), and deletes the pending record.

POST /user/register

Auth: none.

Request body: `email` (required if no phone_number), `phone_number` (required if no email), `password` (required).

Success response: `200` — `{ "message": "OTP sent to user. Please verify to complete registration.", "token": "...", "result": ... }`.

Error responses: `400` missing password/email/phone; `409` user already exists.

Creates/refreshes a `pending_users` row with a hashed password and OTP, then emails (if `email` given) or SMS's (if `phone_number` given) the OTP. Completed via `/user/verify_otp`.

POST /user/register-admin

Auth: none, but requires a hardcoded `passkey` value in the body (`"adnwcbwcqweinov"`).

Request body: `email` (required if no phone_number), `password` (required), `passkey` (required, must match), `phone_number` (required if no email).

Success response: `201` — `{ "message": "Global Admin User registered successfully.", "user_id": <id> }`.

Error responses: 400 missing password/email/phone or invalid passkey; 409 user already exists (based on a pending-users lookup).

Directly creates a users row with is_admin = true (no OTP flow).

POST /user/promote-admin

Auth: requires Bearer JWT; caller must have is_admin=true.

Request body: email or phone_number (at least one required) — identifies the existing user to promote.

Success response: 200 — { "message": "User promoted to admin successfully.", "user_id": <id> }.

Error responses: 401 missing/invalid/expired token; 403 caller is not an admin; 400 missing both email and phone_number; 404 user not found; 409 user is already an admin.

Promotes an existing user (looked up by email or phone number) to global admin by setting is_admin = true, and logs a USER_PROMOTED_TO_ADMIN event. Note: this route is wired in lambdas/user/app.py (elif path == "/user/promote-admin") but is **not** currently declared as an API event in template.yaml's UserAuthFunction resource.

POST /user/login

Auth: none (this endpoint issues the token).

Request body: password (required), plus email or phone_number (at least one required).

Success response: 200 — { "message": "Login successful.", "token": "<JWT, 1yr expiry>", "user_id": <id>, "phone_number": ..., "accounts": [...] }. The JWT payload contains user_id, is_admin, and exp. accounts lists the user's account-holder records (and any pending household-member invitations) across estates.

Error responses: 400 missing credentials; 401 invalid credentials; 403 account deactivated; 500 internal error during login.

Verifies the bcrypt-hashed password, checks is_active, and returns a long-lived JWT plus the user's linked accounts.

POST /user/change_password

Auth: requires Bearer JWT.

Request body: old_password (required), new_password (required).

Success response: 200 — { "message": "Password changed successfully.", "user_id": <id> }.

Error responses: 401 invalid/missing token or old password incorrect; 400 missing fields; 404 user not found.

Changes the authenticated user's own password after verifying the old one.

POST /user/change_type

Auth: requires Bearer JWT with is_admin=true.

Request body: user_id (required) — target user; is_admin (optional, default false) — new admin flag to set.

Success response: 200 — { "message": "User type changed successfully." }.

Error responses: 401 invalid token; 403 caller not admin; 400 missing user_id; 404 user not found.

Sets the is_admin flag on the target user's users row (effectively promotes/demotes global admin status).

POST /user/deactivate

Auth: requires Bearer JWT. Any authenticated user may deactivate themself; an admin may deactivate any user by supplying `user_id`.

Request body: `user_id` (required only when caller `is_admin=true`; otherwise the caller's own token `user_id` is used).

Success response: `200` — `{ "message": "User deactivated successfully." }`.

Error responses: `401` invalid token; `400` admin request missing `user_id`; `404` user not found or already deactivated; `500` internal error.

Sets `is_active = false` on the target user and cascades deactivation to their `account_holders` rows.

POST /user/emailtest

Auth: none.

Request body: `email` (used as recipient) — no validation performed.

Success response: plain string message (not a Lambda-proxy JSON structure) describing the send result — note this handler returns a raw string, not `_success_response`.

Error responses: none formally — exceptions are caught and returned as a string message.

Sends a fixed test email ("Test Email API") to the given address; intended for debugging the email integration.

POST /user/oauth

Auth: none (validates a Google ID token instead of the app's JWT).

Request body: `token` (required) — a Google OAuth ID token.

Success response: `200` — `"Token verified, You can login"` (string, not object) if the token's audience matches one of the configured Google client IDs.

Error responses: `401` invalid Google token.

Verifies a Google Sign-In token via `verify_google_token`, checking the token audience against `GOOGLE_CLIENT_ID_AND`/`GOOGLE_CLIENT_ID_IOS`.

POST /user/submit_report

Auth: requires Bearer JWT.

Request body: `subject` (required), `description` (required).

Success response: `201` — `{ "message": "Report submitted successfully.", "report_id": <id> }`.

Error responses: `401` invalid token; `400` missing subject/description.

Creates a `user_reports` row (with an initial `report_status` of `unresolved`) and emails all active admin users a notification.

POST /user/update_report_status

Auth: requires Bearer JWT with `is_admin=true`.

Request body: `report_id` (required), `status` (required, one of `resolved`, `inprogress`, `unresolved`).

Success response: `200` — `{ "message": "Report status updated successfully." }`.

Error responses: `401` invalid token; `403` caller not admin; `400` missing/invalid `report_id`/`status`; `404` report not found.

Updates the status of an existing user report.

GET /user/accounts/me

Auth: requires Bearer JWT.

Query params: `page` (default 1), `page_size` (default 20, max 100).

Success response: `200` — `{ "accounts": [...], "pagination": {...} }`. Each account row includes `account_holder_id`, `personal_info_id`, `estate_id`, `name`, `expiry`, `contact`, `estate_name`, `account_type_name`, `access_group_name`, `house_address`, `is_active`.

Error responses: `401` invalid token; `400` invalid pagination params.

Returns the authenticated user's own account-holder records across estates.

GET /user/accounts/{user_id}

Auth: requires Bearer JWT with `is_admin=true`.

Path params: `user_id` (required).

Query params: `page`, `page_size` (as above).

Success response: `200` — `{ "accounts": [...], "pagination": {...} }` (subset of fields relative to `/accounts/me`: no `access_group_name`).

Error responses: `401` invalid token; `403` caller not admin; `400` missing `user_id` or invalid pagination.

Returns the account-holder records for a specific user (admin lookup).

GET /user/all

Auth: requires Bearer JWT with `is_admin=true`.

Query params: `page`, `page_size`.

Success response: `200` — `{ "users": [...], "pagination": {...} }`. Each user row: `id`, `email`, `is_active`, `is_admin`, `date_created`, `date_updated`.

Error responses: `401` invalid token; `403` caller not admin; `400` invalid pagination.

Lists all users in the system, paginated.

POST /user/activate

Auth: requires Bearer JWT with `is_admin=true`.

Request body: `user_id` (required).

Success response: `200` — `{ "message": "User reactivated successfully." }`.

Error responses: `401` invalid token; `403` caller not admin; `400` missing `user_id`; `404` user not found or already active.

Sets `is_active = true` on the target user.

POST /user/forgot_password

Auth: none.

Request body: `email` (required, must belong to an active user).

Success response: `200` — `{ "message": "success.", "token": "<JWT carrying user_id>" }`.

Error responses: `400` user not found (does not distinguish "no such user" from other failures — returns a generic message either way).

Generates a reset OTP, stores it with a 10-minute expiry on the `users` row, and emails it, along with a JWT (`{user_id}`) needed for the next step.

POST /user/verify_reset_otp

Auth: requires Bearer JWT (the token issued by `forgot_password`).

Request body: `reset_otp` (required) — OTP the user received by email.

Success response: `200` — `{ "message": "OTP successfully verified." }`.

Error responses: `401` invalid token; `400` missing user_id/OTP; `404` no OTP found; `401` invalid or expired OTP.

Validates the reset OTP and clears it from the `users` row on success.

POST /user/delete_user

Auth: requires Bearer JWT with `is_admin=true`.

Request body: `user_id` (required).

Success response: `200` — `{ "message": "User deleted successfully." }`.

Error responses: `401` invalid token; `403` caller not admin; `400` missing `user_id`; `404` user not found.

Permanently deletes the `users` row for the target user.

POST /user/reset_password

Auth: requires Bearer JWT (typically following `verify_reset_otp`).

Request body: `new_password` (required), `confirm_password` (required, must match `new_password`).

Success response: `200` — `{ "message": "Password changed successfully.", "user_id": <id> }`.

Error responses: `401` invalid token; `500` "Failed to reset password." if `new_password` != `confirm_password` (note: this handler raises a `KeyError`/500 if either body field is absent, since it uses `body["new_password"]` directly rather than `.get`).

Sets a new password for the authenticated user (post-OTP-verification flow).

GET /user/reports

Auth: requires Bearer JWT with `is_admin=true`.

Query params: `page`, `page_size`.

Success response: `200` — `{ "reports": [...], "pagination": {...} }`. Each report: `id`, `subject`, `description`, `date_created`, `user_id`, `reported_by` (email).

Error responses: `401` invalid token; `403` caller not admin; `400` invalid pagination.

Lists all submitted user reports.

GET /user/report_status/{report_id}

Auth: requires Bearer JWT with `is_admin=true`.

Path params: `report_id`.

Success response: `200` — single report object: `report_id`, `status`, `subject`, `description`, `date_created`, `reported_by`.

Error responses: `401` invalid token; `403` caller not admin; `400` neither `report_id` nor `status` provided; `404` report not found.

Fetches the status/details of a single report by ID. (Same handler, `get_report_status`, also serves `/user/report_status?status=...`)

GET /user/report_status

Auth: requires Bearer JWT with `is_admin=true`.

Query params: `status` (required — one of `resolved`, `inprogress`, `unresolved`), `page`, `page_size`.

Success response: `200` — `{ "reports": [...], "pagination": {...} }` filtered by status.

Error responses: `401` invalid token; `403` caller not admin; `400` missing/invalid `status` or bad pagination.

Lists reports filtered by status, paginated. Routed to the same `get_report_status` handler as the path-param variant above; `app.py` distinguishes the two by checking whether the path starts with `/user/report_status/`.

Account API

Base path `/account/...`, implemented in `lambdas/account/` (`app.py` dispatches to `handlers/account.py` for signup/account-level operations, and `handlers/subaccount.py` for household/sub-user management). Nearly every endpoint except `signup` and `signupAdmin` requires a Bearer JWT, and most gate access via the `PrivilegeContext` helpers rather than a flat admin check — meaning an estate-level user with `MANAGE_ACCOUNTS` (or `MANAGE_ALL`) can perform many of these actions, not only global admins.

POST /account/signup

Auth: none.

Request body: `user_id` (required), `estate_id` (required), `house_address` (required), `name` (required), `id_type` (required), `id_num` (required), `contact` (required).

Success response: `201` — `{ "message": "Signup successful.", "account": { account_holder_id, personal_info_id, user_id, estate_id, house_address } }`.

Error responses: `400` missing required fields; `409` user already has an account in this estate; `500` default account type `USER_A` or access group `REGULAR` not found, or insert failure.

Creates a `personal_info` + `account_holders` record for an existing user joining an estate as a regular resident (account type `USER_A`, access group `REGULAR`).

POST /account/admin-signup

Auth: none (no privilege gating in the handler itself — relies on the caller already possessing a valid `user_id`).

Request body: `user_id` (required), `estate_id` (required), `house_address` (required), `name`, `id_type`, `id_num`, `contact`.

Success response: `201` — same shape as `/account/signup`.

Error responses: `400` missing required fields; `409` user already has an account in this estate; `500` insert failure.

Creates a `personal_info` + `account_holders` record with account type `ADMIN` for an estate (used to establish the estate's administrative account holder), marking the account holder `is_active = true` immediately.

POST /account/change_type

Auth: requires Bearer JWT; requires `can_manage_accounts()` privilege (satisfied by `MANAGE_ALL`, `MANAGE_ACCOUNTS`, or global admin) in the estate the target `personal_info` belongs to.

Request body: `personal_info_id` (required), `account_type_id` (required).

Success response: `200` — `{ "message": "Account type updated successfully." }`.

Error responses: `401` invalid token; `403` personal info not found / no account in estate, or insufficient privileges; `400` missing fields; `404` account type or personal info not found.

Changes the `account_type_id` on a `personal_info` row.

POST /account/deactivate

Auth: requires Bearer JWT; requires `can_manage_accounts()` privilege on the account holder's estate.

Request body: `account_holder_id` (required).

Success response: `200` — `{ "message": "Account deactivated successfully." }`.

Error responses: `401` invalid token; `403` account holder not found / no account in estate, or insufficient privileges; `400` missing `account_holder_id`; `404` account not found or already deactivated.

Sets `is_active = false` on an `account_holders` row.

POST /account/activate

Auth: requires Bearer JWT; requires `can_manage_accounts()` privilege on the account holder's estate.

Request body: `account_holder_id` (required).

Success response: `200` — `{ "message": "Account activated successfully." }`.

Error responses: `401` invalid token; `403` account holder not found / no account in estate, or insufficient privileges; `400` missing `account_holder_id`; `404` account not found.

Sets `is_active = true` on an `account_holders` row and sends a push notification ("ACCOUNT ACTIVATED") to the account's user via `notify_user`.

POST /account/update

Auth: requires Bearer JWT. No explicit privilege-context check; instead the underlying SQL restricts the update to rows where the caller is the account's own user, the account's owner (for sub-accounts), or an admin.

Request body: `account_holder_id` (required, target); `name`, `id_type`, `id_num`, `contact`, `access_group_id` (all optional — at least one must be supplied).

Success response: `200` — `{ "message": "Account and related personal info updated successfully." }`.

Error responses: `401` invalid/missing token or user id; `400` missing `account_holder_id` or no fields to update; `404` account holder not found or caller lacks permission to edit it.

Updates fields on the `personal_info` row linked to the given account holder (self, or a sub-account owned by the caller, or any account if admin).

POST /account/sub_user/create

Auth: requires Bearer JWT. Caller must be the parent account's owner, an admin, or hold `can_manage_sub_users()` privilege for the parent account holder.

Request body: `parent_account_holder_id` (required), `email` (used for the invite), `name` (required), `id_type` (required), `id_num` (required), `contact` (required).

Success response: `201` — `{ "message": "Sub-account created successfully.", "sub_account": { email, pending_holder_id, personal_info_id, phone_number } }`.

Error responses: `401` invalid token; `400` missing `parent_account_holder_id` or required fields; `404` parent account holder not found; `403` insufficient privileges to create sub-accounts for this parent; `409` contact already has an account in this estate; `500` on DB config error or unexpected exception.

Creates a `personal_info` row (account type `SUB_USER`) and a `pending_account_holders` invitation entry (with a 30-minute OTP), then emails the invite to the child's email address.

POST /account/sub_user/select_privilege

Auth: requires Bearer JWT. If not admin, requires `can_manage_sub_users()` on the `parent_account_holder_id` supplied in the body.

Request body: `personal_info_id` (required), `privilege_ids` (required, non-empty list), `parent_account_holder_id` (used for the privilege check).

Success response: `201` — `{ "message": "Privileges assigned", "personal_info_id": ..., "privilege_ids": [...] }`.

Error responses: `401` invalid token; `400` missing `personal_info_id/privilege_ids`; `404` personal info not found, or unknown privilege ids; `403` caller cannot manage sub-users (note: if `is_admin` is true and `parent_account_holder_id` privilege lookup is skipped, the handler still requires the `pc` variable to be truthy from a prior call — see implementation caveat below).

Bulk-assigns a set of privileges (`personal_info_privilege_links`) to a sub-user's `personal_info` record. Implementation note: `pc` is only computed inside the `if not is_admin:` branch, so an admin caller hits an `UnboundLocalError` unless `pc` was set by a previous call in the same process — this looks like a latent bug in `assign_selected_privileges_to_personal_info`.

POST /account/sub_user/create_privilege

Auth: requires Bearer JWT. If not admin, requires `can_manage_accounts()` on the estate the `personal_info_id` belongs to.

Request body: `personal_info_id` (required), `privilege_ids` (required — despite the name, a single privilege id).

Success response: `201` — `{ "message": "Privilege linked successfully." }`.

Error responses: `401` invalid token; `400` missing fields; `404` personal info or privilege not found; `403` insufficient privileges; `409` privilege already linked.

Links a single privilege to a `personal_info` record via `personal_info_privilege_links` (idempotent insert).

POST /account/sub_user/verify

Auth: requires Bearer JWT.

Request body: `otp` (required), `personal_info_id` (required), `name` (used for logging only).

Success response: `201` — `{ "message": "Account verified and created successfully.", "user_id": <id> }`.

Error responses: `401` invalid token; `400` missing OTP/personal_info_id; `404` no OTP/pending record found; `401` invalid or expired OTP.

Completes a sub-account invitation: verifies the OTP against `pending_account_holders`, inserts the corresponding `account_holders` row for the (now-authenticated) invited user, and deletes the pending record.

POST /account/sub_user/otp

Auth: requires Bearer JWT (no further privilege check performed in this handler).

Request body: `pending_holder_id` (required), `email` (recipient for the new OTP).

Success response: `200` — `{ "message": "A new OTP has been sent to your email." }`.

Error responses: `401` invalid token.

Regenerates the OTP (30-minute expiry) for a pending sub-account invitation and emails it.

POST /account/sub_user/update_priv

Auth: requires Bearer JWT. If not admin, requires `can_manage_sub_users()` on `parent_account_holder_id`.

Request body: `personal_info_id` (required), `privilege_ids` (required list of ints), `parent_account_holder_id` (used for privilege check).

Success response: `200` — `{ "message": "Privileges updated", "personal_info_id": ..., "privilege_ids": [...] }`.

Error responses: `401` invalid token; `400` missing fields or non-integer `privilege_ids`; `404` personal info not found or unknown privilege ids; `403` caller cannot manage sub-user privileges.

Adds any privilege ids not already linked to the `personal_info` record (existing links are left untouched — this is additive, not a full replace).

POST /account/otp/generate

Auth: requires Bearer JWT. Caller must be the account holder's own user, an admin, or hold `can_manage_accounts()` for the account's estate.

Request body: `account_holder_id` (required).

Success response: `200` — `{ "message": "OTP generated successfully.", "otp": "<6 digits>", "expires_in": 60 }`.

Error responses: `401` invalid token; `400` missing `account_holder_id`; `404` account holder not found; `403` insufficient privileges.

Generates a short-lived (60 second) gate-entry OTP stored on the `account_holders` row.

POST /account/otp/verify

Auth: requires Bearer JWT whose payload identifies a registered tablet device (`gate_id` present and `type == "TABLET"`) — this is a device token, not a regular user token.

Request body: `otp` (required).

Success response: 200 — `{ "message": "OTP verified successfully." }`.

Error responses: 401 missing/invalid token, or caller is not a registered tablet; 404 invalid OTP or no matching account for this estate/gate; 400 OTP has expired or no OTP was generated for the account.

Validates a gate-entry OTP against the estate of the gate identified in the token, and clears it after a successful match.

DELETE /account/delete

Auth: requires Bearer JWT. Caller must be the account's own user, admin, or hold `can_manage_accounts()` on the account's estate.

Request body: `account_holder_id` (required).

Success response: 200 — `{ "message": "Account and related personal info deleted successfully." }`.

Error responses: 401 invalid token; 400 missing `account_holder_id`; 403 account holder not found / no account in estate, or insufficient privileges; 404 personal info not found.

Deletes the `personal_info` row (and cascades to the linked `account_holders` row) for the given account holder.

DELETE /account/sub_user/delete

Auth: requires Bearer JWT. Caller must be the sub-account's owner (parent account's user), admin, or hold `can_manage_sub_users()` on the owning account holder.

Request body: `sub_account_holder_id` (required).

Success response: 200 — `{ "message": "Sub-account deleted (deactivated)" }` (message says "deactivated" but the implementation performs a hard `DELETE`).

Error responses: 401 invalid token; 400 missing `sub_account_holder_id`; 404 sub-account not found; 403 insufficient permission.

Permanently deletes the sub-account's `account_holders`, `personal_info`, and `personal_info_privilege_links` rows.

GET /account/estate/{estate_id}

Auth: requires Bearer JWT; requires `can_manage_accounts()` on the given estate.

Path params: `estate_id` (required).

Query params: `page`, `page_size`.

Success response: 200 — `{ "accounts": [...], "pagination": {...} }`. Each account row includes `account_holder_id`, `user_id`, `house_address`, `is_active`, `personal_info_id`, `full_name`, `contact`, `id_type`, `id_num`, `expiry`, `account_type_name`, `access_group_name`, `estate_id`, `estate_name`.

Error responses: 401 invalid token; 400 missing `estate_id` or bad pagination; 403 no account in estate, or insufficient privileges to view accounts.

Lists all account holders in a given estate (admin/estate-manager view).

GET /account/sub_user/getsub/{user_id}/{estate_id}

Auth: requires Bearer JWT. Caller must be an admin, or `user_id` in the path must equal the caller's own `user_id`.

Path params: `user_id` (required), `estate_id` (required).

Query params: page, page_size (default 20; note the pagination response's total_items/total_pages are not actually computed from a count query and will read 0/1 regardless of result size — a bug in get_sub_accounts).

Success response: 200 — { "sub_accounts": [...], "pagination": { total_items, total_pages, current_page, items_per_page } }. Each row: id, user_id, owner_account_holder_id, house_address, name, contact, id_type, id_num, personal_info_id, email.

Error responses: 401 invalid token or missing user id; 403 caller lacks permission to view another user's sub-accounts; 400 invalid pagination values; 500 database error while fetching.

Lists sub-accounts (household members) owned by a given user within an estate.

Estate API

Base path /estate/..., implemented in lambdas/estate/handlers/estate.py. All endpoints require a Bearer JWT; creation and deletion are restricted to global admins, while updates use estate-level privilege checks (can_manage_all()).

POST /estate/create

Auth: requires Bearer JWT with is_admin=true.

Request body: name (required), location_coordinates (required).

Success response: 201 — { "message": "Estate created successfully. Initial access group created", "estate_id": <id> }.

Error responses: 401 invalid token; 403 caller not admin; 400 missing name/location_coordinates.

Creates a new estates row. A database trigger automatically creates a default REGULAR access group for the new estate.

POST /estate/update

Auth: requires Bearer JWT; requires can_manage_all() privilege for the estate (satisfied by MANAGE_ALL privilege or global admin — note this is stricter than can_manage_accounts() used elsewhere).

Request body: id (required, estate id); name and/or location_coordinates (at least one required).

Success response: 200 — { "message": "Estate updated successfully.", "estate_id": <id> }.

Error responses: 401 invalid token; 400 missing id or no update fields; 403 no account in estate, or insufficient privileges; 404 estate not found.

Updates the estate's name and/or location_coordinates (fields not supplied are left unchanged via COALESCE).

DELETE /estate/delete

Auth: requires Bearer JWT with is_admin=true.

Request body: id (required, estate id).

Success response: 200 — { "message": "Estate deleted successfully." }.

Error responses: 401 invalid token; 403 caller not admin; 400 missing id; 404 estate not found.

Permanently deletes an estates row.

GET /estate/all

Auth: requires Bearer JWT (any authenticated user; no admin/privilege check performed).

Query params: `page` (default 1), `page_size` (default 20, max 100).

Success response: `200` — `{ "estates": [...], "pagination": {...} }`. Each estate: `id`, `name`, `location_coordinates`, `date_created`, `date_updated`.

Error responses: `401` invalid token; `400` invalid pagination params.

Lists all estates in the system, paginated.

GET /estate/{id}

Auth: requires Bearer JWT (any authenticated user; no further privilege check).

Path params: `id` (required, estate id).

Success response: `200` — `{ "estate": { id, name, location_coordinates, date_created, date_updated } }`.

Error responses: `401` invalid token; `400` missing estate id; `404` estate not found.

Retrieves a single estate's details by ID.

All endpoints below are implemented as AWS Lambda functions behind API Gateway (SAM `template.yaml`), routed by simple `if/elif` dispatch in each module's `app.py` on `httpMethod` + `path`. Every response is wrapped with permissive CORS headers (`Access-Control-Allow-Origin: *`) via `cors_wrap`.

Unless noted otherwise, "requires Bearer JWT" means the handler calls `verify_token(event)`, which reads the `Authorization: Bearer <token>` header, decodes it with `JWT_SECRET` (HS256), and returns `401` if the header is missing or the token is invalid/expired. The decoded payload typically carries `user_id` and `is_admin`; gate-controller devices instead carry `gate_id` and `type: "GATE"`.

Privilege checks are performed via `PrivilegeContext` helpers in `shared/python/utils.py` (e.g. `get_user_privileges_for_`
`get_user_privileges_for_gate`, `get_user_privileges_for_access_group`). `is_admin=True` in the JWT always satisfies any privilege check. Otherwise the user's privileges are resolved from `account_types`/`privileges` tables for their account holder in the relevant estate.

Gate API

Routes for gate CRUD/lifecycle management, vehicle-access lookups used by gate controllers and app clients, and estate dashboard/system-monitoring endpoints — all served by the single `GateFunction` Lambda (`lambdas/gate/app.py` → `handlers/gate.py` and `handlers/dashboard_handlers.py`).

POST /gate/create

Auth: Requires Bearer JWT. Caller must have an account holder in `estate_id` and pass `can_manage_gates()` (privilege `MANAGE_GATES` or `MANAGE_ALL`, or `is_admin=true`).

Request body: - `name` (string, required) — gate name - `estate_id` (integer, required) — estate the gate belongs to - `opening_time` (string, optional) — gate open time; if provided together with `closing_time`, both are stored - `closing_time` (string, optional) — gate close time

Success response: `201` — `{"message": "Gate created successfully.", "gate_id": <id>}`

Error responses: - `400` — missing `name` or `estate_id` - `403` — caller has no account in the estate, or lacks gate-management privileges

Creates a new gate row for an estate and writes a `GATE_CREATION` log entry.

POST /gate/update

Auth: Requires Bearer JWT. Resolves privileges via the gate's estate (`get_user_privileges_for_gate`); requires `can_manage_gates()`.

Request body: - `id` (integer, required) — gate ID to update - `name` (string, optional) - `opening_time` (string, optional) - `closing_time` (string, optional)

Unset optional fields are preserved (`COALESCE` against existing values).

Success response: `200` — `{"message": "Gate updated successfully.", "gate_id": <id>}`

Error responses: - `400` — missing `id` - `403` — no account in the gate's estate, or insufficient privileges - `404` — gate not found

Updates a gate's name and/or opening/closing time, logging `GATE_UPDATE`.

POST /gate/heartbeat

Auth: Requires Bearer JWT issued to a gate controller — the token payload must include a truthy `gate_id` and `type == "GATE"`, otherwise `401 Unauthorized`.

Request body: - `gate_id` (integer, required) — gate whose heartbeat row is updated (note: the value used in the `UPDATE` is the body's `gate_id`, not the token's) - `SNR` (optional) — signal-to-noise ratio reading - `RSSI` (optional) — signal strength reading

Success response: `200` — `{"message": "Controller heartbeat updated successfully.", "gate_id": <id>}`

Error responses: - `401` — token is not a valid gate-controller token - `404` — gate not found

Called periodically by a gate controller device to record `last_controller_heartbeat`, `snr`, and `rssi`; logs `GATE_CONTROLLER_HEARTBEAT`.

POST /gate/activate

Auth: Requires Bearer JWT; requires `can_manage_gates()` for the gate's estate (or `is_admin`).

Request body: `id` (integer, required) — gate ID

Success response: `200` — `{"message": "Gate activated successfully.", "gate_id": <id>}`

Error responses: - `400` — missing `id` - `403` — no account in the gate's estate, or insufficient privileges - `404` — gate not found

Sets `active = true` on the gate and logs `GATE_ACTIVATION`.

POST /gate/deactivate

Auth: Same as activate — requires Bearer JWT and `can_manage_gates()`.

Request body: `id` (integer, required) — gate ID

Success response: `200` — `{"message": "Gate deactivated successfully.", "gate_id": <id>}`

Error responses: - `400` — missing `id` - `403` — no account in the gate's estate, or insufficient privileges - `404` — gate not found

Sets `active = false` on the gate and logs `GATE_DEACTIVATION`.

POST /gate/vehicle_arrival/callback

Auth: Requires Bearer JWT issued to a gate controller (`gate_id` + `type == "GATE"` in payload), else `401`.

Request body: - `license_plate_number` (string, required) - `status` (string, required) — must be `"entry"` or `"exit"` - `timestamp` (string, optional) — human-readable timestamp used only in log/notification text

Success response: `200` —

```
{
  "message": "Gate opened for vehicle entry|exit",
  "person_name": "...",
  "type": "guest" | "account_holder",
  "status": "entry" | "exit",
  "guest_status": "CHECKED_IN" | "CHECKED_OUT" | null
}
```

Error responses: - `400` — missing `license_plate_number`; missing/invalid `status`; vehicle not found for a guest/account holder in the gate's estate; guest has no valid entry window; entry window has invalid dates; current time outside entry window; guest record could not be updated - `401` — not a valid gate-controller token

Called by a gate controller when a vehicle arrives or departs. Looks up the vehicle's owner (guest or account holder) by license plate scoped to the gate's estate. For guests entering, validates the guest's `entry_window` (stored in `guests.other_info`), then clears `otp`/`entry_window` and sets guest status to `CHECKED_IN`; exits set `CHECKED_OUT` without a time check. Account holders bypass the window check entirely. Sends a push notification (`notify_user`) to the relevant user, writes a log entry, and inserts a row into `accesslog`.

POST /gate/email_user

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `400` (not `403`).

Request body: - `message` (string) — email body text - `gate_name` (string) — gate whose associated estate users will be emailed

Success response: `200` — `{"message": "Gate Update sent successfully.", "gate_name": "..."}`

Error responses: - `400` — caller is not an admin - `404` — no gate matching `gate_name`, or no user emails found for its estate

Looks up all account-holder users in the gate's estate and emails each one (via `send_email`, an external messaging API call) with the given message under subject "Gate Access System Update". Per-recipient send failures are caught and skipped, not surfaced as an overall error.

DELETE /gate/delete

Auth: Requires Bearer JWT; requires `can_manage_gates()` for the gate's estate.

Request body: `id` (integer, required) — gate ID

Success response: `200` — `{"message": "Gate deleted successfully."}`

Error responses: - `400` — missing `id` - `403` — no account in the gate's estate, or insufficient privileges - `404` — gate not found

Deletes the gate row and logs `GATE_DELETION`.

GET /gate/all

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Query params: `page` (default 1), `page_size` (default 20, max 100)

Success response: `200` —

```
{
  "gates": [{ "id", "name", "estate_id", "active", "last_controller_heartbeat", "date_created", "date_u
  "pagination": { "page", "page_size", "total", "pages" }
}
```

Error responses: - `400` — invalid `page`/`page_size` - `403` — caller is not an admin

Lists all gates across all estates, paginated, ordered by newest first.

GET /gate/estate/{estate_id}

Auth: Requires Bearer JWT; requires an account in the estate and `can_manage_gates()`.

Path params: `estate_id` (required) **Query params**: `page` (default 1), `page_size` (default 20, max 100)

Success response: `200` — same shape as `/gate/all`, scoped to the estate

Error responses: - `400` — missing `estate_id` path param, or invalid pagination params - `403` — no account in the estate, or insufficient privileges

Lists gates belonging to a specific estate, paginated.

GET /gate/{id}

Auth: Requires Bearer JWT; requires `can_manage_gates()` for the gate's estate.

Path params: `id` (required) — gate ID

Success response: `200` — `{"gate": {...}}` with the same gate fields as the list endpoints

Error responses: - `400` — missing gate ID - `403` — no account in the gate's estate, or insufficient privileges - `404` — gate not found

Fetches a single gate by ID.

GET /gate/controller/vehicles

Auth: Requires Bearer JWT issued to a gate controller (`gate_id` + `type == "GATE"` in payload); no privilege check beyond that.

Params: none (gate is taken from the token)

Success response: `200` — `{"vehicles": [...]}` (full `vehicles.*` rows)

Error responses: `401` — not a valid gate-controller token

Returns all vehicles allowed through the gate's linked access groups, ignoring time-window restrictions. Intended for gate-controller hardware polling.

GET /gate/controller/timed_vehicles

Auth: Same as above — gate-controller JWT only.

Params: none

Success response: `200` — `{"vehicles": [...]}` including `pi.expiry`

Error responses: `401` — not a valid gate-controller token

Returns vehicles allowed through the gate's access groups, applying the gate's/link's `opening_time`/`closing_time` window (including windows that cross midnight) and, for guests, an `entry_window` stored in `other_info`.

GET /gate/vehicles/{gate_id}

Auth: Requires Bearer JWT; requires `can_manage_gates()` for the gate's estate.

Path params: `gate_id` (required)

Success response: `200` — `{"vehicles": [...]}`

Error responses: - `400` — missing `gate_id` - `403` — no account in the gate's estate, or insufficient privileges

Same query as `/gate/controller/vehicles` (no time check) but for authenticated app users/admins rather than controllers.

GET /gate/timed_vehicles/{gate_id}

Auth: Requires Bearer JWT; requires `can_manage_gates()` for the gate's estate.

Path params: `gate_id` (required) **Query params**: `page` (default 1), `page_size` (default 20, max 100)

Success response: `200` —

```
{
  "vehicles": [...],
  "pagination": { "page", "page_size", "total", "pages" }
}
```

Error responses: - `400` — missing `gate_id`, or invalid pagination params - `403` — no account in the gate's estate, or insufficient privileges

Paginated version of the timed vehicle-access lookup, for authenticated users.

POST /system/metrics

Auth: Requires Bearer JWT issued to a gate controller (`gate_id` + `type == "GATE"` in payload).

Request body: - `timestamp` (string, required) — ISO-8601 device timestamp - `gate_id` (optional) — if present, must match the token's `gate_id` or the request is rejected - `temperature_celsius` (required) - `total_cameras` (required) - `online_cameras` (required) - `storage_used_gb` (required) - `storage_total_gb` (required) - `running_services` (object, optional, stored as JSON) - `offline_camera_ids` (array, optional, stored as JSON)

Success response: `201` — `{"message": "Metrics received successfully", "metric_id": <id>}`

Error responses: - `400` — missing/mismatched `timestamp`/`gate_id`, invalid timestamp format, or missing required numeric fields - `401` — not a valid gate-controller token - `500` — database insert failure

Ingests a metrics snapshot from a gate controller into `system_metrics` (temperature, camera counts, storage, running services, offline cameras) and keeps an in-memory copy for the current invocation.

GET /system/latest_metrics

Auth: Requires Bearer JWT (any authenticated caller — no admin/privilege check in this handler).

Query params: `gate_id` — required unless the caller's own token already carries a `gate_id` (gate-controller tokens), in which case it defaults to that.

Success response: `200` —

```
{
  "data_received_at": "<ISO timestamp>",
  "metrics": { "...system_metrics row fields...", "running_services": [...], "number_running_services":
}
```

Error responses: - `400` — missing or non-integer `gate_id` - `404` — no metrics stored for the gate

Returns the most recent `system_metrics` row for a gate.

GET /system/temperature

Auth: None — this handler does not call `verify_token`; it is reachable without a token.

Query params: `start_date`, `end_date` (ISO-8601, required); `gate_id` (optional filter); `granularity` (`hourly`, `daily`, or default raw/per-record, optional)

Success response: `200` — `{"trend_data": [{"timestamp", "temperature_celsius"}, ...]}`, bucketed and averaged per the requested granularity

Error responses: `400` — missing `start_date`/`end_date`, or non-ISO-8601 dates

Returns historical temperature readings from `system_metrics` for charting.

GET /cameras/downtime

Auth: None — this handler does not call `verify_token`.

Query params: `gate_id`, `start_date`, `end_date` (all required, ISO-8601 dates)

Success response: `200` — `{"offline_events": [{"camera_id": <offline_camera_ids value>, "start_time": "..."}]}`

Error responses: `400` — missing params, or invalid date/`gate_id` format

Returns the raw `offline_camera_ids` values recorded per metrics sample within the time window for a gate (the code contains a larger commented-out implementation that would compute actual downtime intervals; the live version just echoes each sample's offline-camera data with its timestamp).

GET /system/storage

Auth: None — this handler does not call `verify_token`.

Query params: `start_date`, `end_date` (required, ISO-8601); `gate_id` (optional filter); `granularity` (`hourly`, `daily`, or raw, optional)

Success response: `200` — `{"trend_data": [{"timestamp", "storage_used_gb", "storage_total_gb", "storage_used_pct"}, ...]}`

Error responses: `400` — missing `start_date`/`end_date`, or invalid dates

Returns historical storage usage from `system_metrics`, bucketed/averaged by granularity.

GET /system/access

Auth: None — this handler does not call `verify_token`.

Query params: `gate_id`, `start_date`, `end_date` (all required)

Success response: `200` —

```
{
  "gate_events": {
    "events": [{"license_plate_number", "event", "event_count", "timestamps": [...]}, ...],
    "totals": {"entry": n, "exit": n}
  }
}
```

Error responses: 400 — missing params, or invalid gate_id/date format

Aggregates accesslog entries (entries/exits) for a gate within a time window, grouped by license plate and event type, plus overall entry/exit totals.

GET /system/videos

Auth: Requires Bearer JWT with is_admin=true; otherwise 403.

Query params: - start_date, end_date (required, YYYY-MM-DD) - camera_id (required, matched case-insensitively as a substring of the S3 object key) - page (default 1), limit (default 20)

Success response: 200 —

```
{
  "camera": "...",
  "total_found": n,
  "current_page": n,
  "limit": n,
  "has_more": true|false,
  "videos": [{"filename", "url", "last_modified"}, ...]
}
```

Error responses: - 400 — non-integer page/limit, or missing start_date/end_date/camera_id - 403 — caller is not an admin - 500 — unexpected error while scanning S3

Lists recorded video/image objects for a camera within a date range by scanning per-day S3 prefixes (lp-images/<date>/) in the itc-lpr-bucket bucket, then generates presigned S3 URLs (1-hour expiry) only for the current page of results.

Access Group API

Routes for managing access groups — named collections used to control which vehicles/personal-info records are allowed through which gates. Served by AccessGroupFunction (lambdas/access_group/app.py → handlers/access_group.py).

POST /access_group/create

Auth: Requires Bearer JWT; requires an account in estate_id and can_manage_access_groups() privilege (MANAGE_ACCESS or MANAGE_ALL, or is_admin).

Request body: - name (string, required) — stored upper-cased - estate_id (integer, required)

Success response: 201 — {"message": "Access Group created successfully.", "access_group_id": <id>}

Error responses: - 400 — missing name or estate_id - 403 — no account in the estate, or insufficient privileges

Creates an access group scoped to an estate and logs ACCESS_GROUP_CREATION.

POST /access_group/update

Auth: Requires Bearer JWT; privileges resolved from the access group's estate; requires can_manage_access_groups().

Request body: - id (integer, required) — access group ID - name (string, required) — new name, stored upper-cased

Success response: 200 — {"message": "Access Group updated successfully.", "access_group_id": <id>}

Error responses: - `400` — missing `id` or `name` - `403` — access group not found / no account in its estate, or insufficient privileges - `404` — access group not found (post-privilege-check race)

Renames an access group and logs `ACCESS_GROUP_UPDATE`.

DELETE /access_group/delete

Auth: Requires Bearer JWT; requires `can_manage_access_groups()` for the access group's estate.

Request body: `id` (integer, required)

Success response: `200` — `{"message": "Access Group deleted successfully."}`

Error responses: - `400` — missing `id` - `403` — access group not found / no account in its estate, or insufficient privileges - `404` — access group not found

Deletes an access group and logs `ACCESS_GROUP_DELETION`.

GET /access_group/all

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Query params: `page` (default 1), `page_size` (default 20, max 100)

Success response: `200` —

```
{
  "access_groups": [{ "id", "name", "date_created", "date_updated" }, ...],
  "pagination": { "page", "page_size", "total", "pages" }
}
```

Error responses: - `400` — invalid pagination params - `403` — caller is not an admin

Lists all access groups across all estates.

GET /access_group/estate/{id}

Auth: Requires Bearer JWT; requires an account in the estate and `can_manage_access_groups()`.

Path params: `id` (required) — estate ID **Query params**: `page` (default 1), `page_size` (default 20, max 100)

Success response: `200` — same shape as `/access_group/all` (rows also include `estate_id`), scoped to the estate

Error responses: - `400` — missing estate ID, or invalid pagination params - `403` — no account in the estate, or insufficient privileges

Lists access groups belonging to one estate.

GET /access_group/{id}

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403` (note: the handler's error message says "Only admins can delete estates" despite this being a GET-by-id lookup — apparent copy/paste artifact in the code).

Path params: `id` (required) — access group ID

Success response: `200` — `{"access_group": {"id", "name", "date_created", "date_updated"}}`

Error responses: - `400` — missing access group ID - `403` — caller is not an admin - `404` — access group not found

Fetches a single access group by ID.

Account Type API

Routes for managing account types (roles used to bundle default privileges for personal-info records). Served by `AccountTypeFunction` (`lambdas/account_type/app.py` → `handlers/account_type.py`). All mutating endpoints are global-admin-only; there is no estate scoping for account types.

POST /account_type/create

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Request body: `name` (string, required)

Success response: `201` — `{"message": "Account Type created successfully.", "account_type_id": <id>}`

Error responses: - `400` — missing `name` - `403` — caller is not an admin

Creates a new account type and logs `ACCOUNT_TYPE_CREATED`.

POST /account_type/update

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Request body: - `id` (integer, required) - `name` (string, required)

Success response: `200` — `{"message": "Account Type updated successfully.", "account_type_id": <id>}`

Error responses: - `400` — missing `id` or `name` - `403` — caller is not an admin - `404` — account type not found

Renames an account type (also bumps `date_updated`) and logs `ACCOUNT_TYPE_UPDATED`.

DELETE /account_type/delete

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Request body: `id` (integer, required)

Success response: `200` — `{"message": "Account Type deleted successfully."}`

Error responses: - `400` — missing `id` - `403` — caller is not an admin - `404` — account type not found

Deletes an account type and logs `ACCOUNT_TYPE_DELETED` (the log message text says "created", which appears to be a copy/paste bug in the code).

GET /account_type/all

Auth: Requires Bearer JWT (any authenticated caller — no admin check in this handler).

Query params: `page` (default 1), `page_size` (default 20, max 100)

Success response: `200` —

```
{
  "account_types": [{ "id", "name", "date_created", "date_updated" }, ...],
  "pagination": { "page", "page_size", "total", "pages" }
}
```

Error responses: `400` — invalid pagination params

Lists all account types, paginated.

GET /account_type/{id}

Auth: Requires Bearer JWT only (no admin/privilege check).

Path params: `id` (required) — account type ID

Success response: `200` — `{"account_type": {"id", "name", "date_created", "date_updated"}}`

Error responses: - `400` — missing account type ID - `404` — account type not found

Fetches a single account type by ID.

Privilege API

Routes for managing the master list of privilege names (e.g. `MANAGE_GATES`, `MANAGE_ACCESS`) that can be attached to account types or individual personal-info records. Served by `PrivilegeFunction` (`lambdas/privilege/app.py` → `handlers/privilege.py`). Every endpoint in this module is global-admin-only.

POST /privilege/create

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Request body: `name` (string, required) — stored upper-cased

Success response: `201` — `{"message": "Privilege created successfully.", "privilege_id": <id>}`

Error responses: - `400` — missing `name` - `403` — caller is not an admin

Creates a new privilege row and logs `PRIVILEGE_CREATION`.

POST /privilege/update

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Request body: - `id` (integer, required) - `name` (string, required) — stored upper-cased

Success response: `200` — `{"message": "Privilege updated successfully.", "privilege_id": <id>}`

Error responses: - `400` — missing `id` or `name` - `403` — caller is not an admin - `404` — privilege not found

Renames a privilege (bumps `date_updated`) and logs `PRIVILEGE_UDATE` (sic — typo in the log type string in the source).

DELETE /privilege/delete

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Request body: `id` (integer, required)

Success response: `200` — `{"message": "Privilege deleted successfully."}`

Error responses: - `400` — missing `id` - `403` — caller is not an admin - `404` — privilege not found

Deletes a privilege and logs `PRIVILEGE_DELETION`.

GET /privilege/all

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Query params: `page` (default 1), `page_size` (default 20, max 100)

Success response: `200` —

```
{
  "privileges": [{ "id", "name", "date_created", "date_updated" }, ...],
  "pagination": { "page", "page_size", "total", "pages" }
}
```

Error responses: - `400` — invalid pagination params - `403` — caller is not an admin

Lists all privileges, paginated.

GET /privilege/{id}

Auth: Requires Bearer JWT with `is_admin=true`; otherwise `403`.

Path params: `id` (required) — privilege ID

Success response: `200` — `{"privilege": {"id", "name", "date_created", "date_updated"}}`

Error responses: - `400` — missing privilege ID - `403` — caller is not an admin - `404` — privilege not found

Fetches a single privilege by ID.

These four modules manage the core domain objects of the gate-access system: guests (visitors invited by an account holder), vehicles associated with a person, events hosted by an account holder, and event registrations (guests attending a specific event). All four lambdas share the same request-dispatch pattern: `app.py` parses the path/method and calls a handler function in `handlers/`, which authenticates the caller, checks privileges, executes the database operation, and returns a JSON response wrapped with CORS headers.

Unless noted otherwise, all endpoints require a `Bearer <JWT>` token in the `Authorization` header. The token is decoded via `verify_token` (`shared/python/middleware.py`), which yields a payload containing `user_id` and, optionally, `is_admin`. Authorization beyond "is this a valid token" is then enforced per-endpoint using `PrivilegeContext` (`shared/python/utils.py`), built from `get_user_privileges(cur, user_id, account_holder_id, is_admin)`. Admins (`is_admin: true` in the JWT) bypass all privilege checks. All error responses share the shape `{"error": "<message>"}`; all success responses are raw JSON objects (no envelope) with an HTTP success status code.

Guest API

Routes are prefixed `/guest/...` and implemented in `lambdas/guest/app.py` / `lambdas/guest/handlers/guest.py`. Guests are visitors registered under an account holder; a guest optionally has a vehicle, and gets either a car-lane invite or a 6-digit OTP for pedestrian entry.

POST /guest/create

Registers a new guest (with a `personal_info` record) under an account holder, optionally with a vehicle. If no vehicle is supplied, a unique 6-digit OTP is generated. Sends an SMS to the guest with either the OTP or the car-lane invite details.

- **Auth**: Bearer JWT. Requires `can_create_guests()` on the target `account_holder_id` (i.e. `MANAGE_ALL`, `MANAGE_GUESTS`, or `CREATE_GUESTS`). If a vehicle is included, also requires `can_create_vehicles()`.
- **Request body**:

  - `account_holder_id` (required) — account holder the guest is created under
  - `name` (required)
  - `id_type` (required)
  - `id_num` (required)
  - `contact` (required) — phone number, used for SMS
  - `expiry` (required) — ISO datetime, end of entry window
  - `start` (optional) — ISO datetime, defaults to now
  - `other_info` (optional) — object merged with generated `entry_window`/`otp`/`purpose` fields
  - `license_plate_number`, `make`, `model`, `color` (optional) — if `license_plate_number` is given, `make`/`model`/`color` become required
- **Success response**: `201` — `{"message": "...", "guest_id": <int>, "personal_info_id": <int>}`
- **Errors**: `400` missing required fields; `400` if vehicle fields are incomplete; `400` "No estate found for this account holder"; `403` insufficient privileges to create a guest or (separately) a vehicle; `500` on unexpected DB/other errors.

POST /guest/create_associate

Same as `/guest/create` but registers a guest with an explicit `account_type_id` (an "associate" — a non-GUEST account type), otherwise following identical vehicle/OTP logic and SMS notification.

- **Auth**: Bearer JWT. Requires `can_create_guests()` on `account_holder_id`; requires `can_create_vehicles()` if a vehicle is included.
- **Request body**: same as `/guest/create` plus `account_type_id` (required).
- **Success response**: `201` — `{"message": "...", "guest_id": <int>, "personal_info_id": <int>}`
- **Errors**: same as `/guest/create`, with `account_type_id` also validated as required.

POST /guest/update

Updates a guest's `personal_info` and vehicle. Can add, update, or remove the associated vehicle depending on which fields are supplied, regenerating an OTP if the vehicle is removed or was never present.

- **Auth**: Bearer JWT. Requires `can_manage_or_own_guests()` — i.e. `MANAGE_ALL`/`MANAGE_GUESTS`, or `CREATE_GUESTS` plus being the same person as the creator. Vehicle create/update/remove additionally requires `can_create_vehicles()`.
- **Request body**:
  - `guest_id` (required)
  - `account_holder_id` (optional) — if supplied, must match the guest's existing creator or a `403` is returned
  - `name`, `id_type`, `id_num`, `contact`, `expiry`, `start`, `other_info` (all optional, only supplied fields are updated)
  - `license_plate_number`, `make`, `model`, `color` (optional; if `license_plate_number` is present, the other three are required)
- **Success response**: `200` — `{"message": "...", "guest_id": <int>, "has_vehicle": <bool>, "otp_generated": <bool>, "entry_window_updated": <bool>}`
- **Errors**: `400` missing `guest_id`; `400` incomplete vehicle fields; `404` guest not found; `403` guest doesn't belong to given `account_holder_id`, or insufficient privileges to update the guest or its vehicle; `500` on unexpected errors.

POST /guest/verify_otp

Verifies a guest's OTP against the current time window. On success, clears the `otp`/`entry_window` from the guest record (single-use). Intended for gate tablets rather than app users.

- **Auth**: Bearer JWT, but the token must belong to a registered **TABLET** device — the payload must contain `gate_id` and `type == "TABLET"`, not a normal user token.
- **Request body**: `otp` (required)
- **Success response**: `200` — `{"message": "OTP matched successfully", "guest_name": <str>, "guest_id": <int>, "inviter_user_id": <int>}`
- **Errors**: `401` if not a tablet token; `400` missing `otp`; `400` invalid OTP (no match for this gate's estate); `400` no valid entry window found; `400` invalid date format; `400` OTP outside its valid time window.

POST /guest/send_otp

Generates and sends a new OTP for an existing guest, replacing any previous OTP/entry window.

- **Auth**: Bearer JWT. Requires `can_manage_guests()`, OR the caller must be the guest's creator (checked directly against `guests.creator_id` / `account_holders.user_id`).
- **Request body**: `account_holder_id` (required), `guest_id` (required), `expiry` (required), `start` (optional, defaults to now)
- **Success response**: `200` — `{"message": "OTP generated and sent successfully.", "guest_name": <str>}`
- **Errors**: `400` missing required fields; `400` "No estate found for this account holder"; `403` not privileged/not the creator; `404` guest not found; `400` if the OTP update fails.

DELETE /guest/delete

Deletes a guest and cascades deletion of the associated `personal_info` (and vehicle, via DB cascade).

- **Auth**: Bearer JWT. Requires `can_manage_or_own_guests()`.
- **Request body**: `guest_id` (required)
- **Success response**: `200` — `{"message": "Guest and personal info deleted successfully."}`
- **Errors**: `400` missing `guest_id`; `404` guest not found (or personal info not found); `400` "No estate found for this account holder"; `403` insufficient privileges.

GET /guest/account_holder/{account_holder_id}

Lists guests created by the given account holder, paginated.

- **Auth**: Bearer JWT. Requires `can_manage_or_own_guests()` for `account_holder_id`.
- **Request**: path param `account_holder_id`; query params `page` (default 1), `page_size` (default 20, max 100)
- **Success response**: `200` — `{"guests": [...], "pagination": {"page", "page_size", "total", "pages"}}`
- **Errors**: `400` missing `account_holder_id`; `400` invalid `page`/`page_size`; `400` "No estate found…"; `403` insufficient privileges.

GET /guest/estate/{estate_id}

Lists all guests created within an estate (across all account holders), paginated.

- **Auth**: Bearer JWT. Requires admin, or `MANAGE_ALL`/`MANAGE_GUESTS` privilege within `estate_id` (checked via a direct DB query, not `PrivilegeContext`).
- **Request**: path param `estate_id`; query params `page`, `page_size`
- **Success response**: `200` — `{"guests": [...], "pagination": {...}}` (each guest includes `creator_name`/`creator_id`)
- **Errors**: `400` missing `estate_id`; `400` invalid pagination params; `403` insufficient privileges.

GET /guest/{guest_id}

Retrieves full details of a single guest, including estate, account type, and access group names.

- **Auth**: Bearer JWT. Requires `can_manage_or_own_guests()` (resolved from the guest's creator).
- **Request**: path param `guest_id`
- **Success response**: `200` — `{"guest": [{...}]}` (a single-element array from `process_db_results`)
- **Errors**: `400` missing `guest_id`; `404` guest not found; `400` "No estate found…"; `403` insufficient privileges.

Vehicle API

Routes are prefixed `/vehicle/...` and implemented in `lambdas/vehicle/app.py` / `lambdas/vehicle/handlers/vehicle.p`
Vehicles belong to a `personal_info` record (an account holder or a guest); license plates must be unique per estate.

POST /vehicle/create

Registers a new vehicle for a person (account holder or guest), enforcing a unique license plate per estate.

- **Auth**: Bearer JWT. Requires `can_manage_vehicle_for_person(cur, personal_info_id)` — i.e. `MANAGE_ALL`/`MANAGE_VEHICLES`, or `CREATE_VEHICLES` plus being the vehicle owner or the creator of the guest that owns it.
- **Request body**: `personal_info_id` (required), `license_plate_number` (required), `make` (required), `model` (required), `color` (required)
- **Success response**: `201` — `{"message": "Vehicle registered successfully.", "vehicle_id": <int>}`
- **Errors**: `400` missing required fields; `400` personal info not found; `400` "No estate found…"; `403` insufficient privileges; `409` license plate already registered at this estate.

POST /vehicle/update

Updates a vehicle's plate/make/model/color. Only supplied fields are changed (`COALESCE`).

- **Auth**: Bearer JWT. Requires `can_manage_vehicle_for_person(cur, vehicle_personal_info_id)`.
- **Request body**: `vehicle_id` (required), `personal_info_id` (required — used only for the initial required-field check, not re-verified against the vehicle's actual owner), `license_plate_number`, `make`, `model`, `color` (all optional)
- **Success response**: `200` — `{"message": "Vehicle updated successfully."}`
- **Errors**: `400` missing `vehicle_id`/`personal_info_id`; `404` vehicle not found; `400` personal info not found; `400` "No estate found…"; `403` insufficient privileges; `500` if the update fails to affect a row.

DELETE /vehicle/delete

Deletes a vehicle.

- **Auth**: Bearer JWT. Requires `can_manage_vehicle_for_person(cur, vehicle_personal_info_id)`.
- **Request body**: `vehicle_id` (required)
- **Success response**: `200` — `{"message": "Vehicle deleted successfully."}`
- **Errors**: `400` missing `vehicle_id`; `404` vehicle not found; `400` personal info not found; `400` "No estate found…"; `403` insufficient privileges; `500` if the delete fails.

GET /vehicle/{vehicle_id}

Retrieves details for a single vehicle, including the owner's name and contact.

- **Auth**: Bearer JWT. Requires `can_view_vehicle_for_person(cur, vehicle_personal_info_id)` (management privilege, or direct ownership of the `personal_info` record).
- **Request**: path param `vehicle_id`

- **Success response**: 200 — `{"vehicle": [{"id", "license_plate_number", "make", "model", "color", "owner_name", "contact"}]}`
- **Errors**: 400 missing `vehicle_id`; 404 vehicle not found; 400 personal info not found; 400 "No estate found…"; 403 insufficient privileges.

GET /vehicle/person/{personal_info_id}

Lists all vehicles belonging to a given person, paginated.

- **Auth**: Bearer JWT. Requires `can_view_vehicle_for_person(cur, personal_info_id)`.
- **Request**: path param `personal_info_id`; query params `page`, `page_size`
- **Success response**: 200 — `{"vehicles": [...], "pagination": {"page", "page_size", "total", "pages"}}`
- **Errors**: 400 missing `personal_info_id`; 400 invalid pagination params; 400 personal info not found; 400 "No estate found…"; 403 insufficient privileges.

Event API

Routes are prefixed `/event/...` and implemented in `lambdas/event/app.py` / `lambdas/event/handlers/event.py`. Events are hosted by an account holder and carry a signed JWT-based invite link that guests can use (via the Event Register API) to self-register.

POST /event/create

Creates a new event and immediately generates its invite link (internally calling the same logic as `/event/invite/generate`).

- **Auth**: Bearer JWT. Requires `can_manage_events()` or `can_create_events()` on `creator_id`.
- **Request body**: `creator_id` (required), `name` (required), `max_num_of_attendees` (required), `start_time` (required), `end_time` (required)
- **Success response**: 201 — `{"message": "Event created successfully.", "event_id": <int>, "invite_url": <str>}`
- **Errors**: 400 missing required fields; 400 "No estate found…"; 403 insufficient privileges; 500 "Event created but failed to generate invite link." if invite generation fails after the event insert.

POST /event/update

Updates an event's details. Only supplied fields are changed.

- **Auth**: Bearer JWT. Requires `can_manage_or_own_events()` (manage privilege, or being the event's own creator with create privilege).
- **Request body**: `event_id` (required), `name`, `max_num_of_attendees`, `start_time`, `end_time` (all optional)
- **Success response**: 200 — `{"message": "Event updated successfully."}`
- **Errors**: 400 missing `event_id`; 404 event not found; 400 "No estate found…"; 403 insufficient privileges; 500 if the update fails.

POST /event/invite/generate

Generates a signed JWT invite link (24-hour expiry) for an event, embedding event/creator/estate metadata. The link is `https://gate-access-web.vercel.app/invite?token=<jwt>`.

- **Auth**: Bearer JWT. Requires `can_manage_events()` or `can_create_events()` on the event's creator.
- **Request body**: `event_id` (required)
- **Success response**: 200 — `{"message": "...", "invite_url": <str>, "expires_in_hours": 24}`

- **Errors**: 400 missing event_id; 404 event not found; 400 "No estate found…"; 403 insufficient privileges.

DELETE /event/delete

Deletes an event.

- **Auth**: Bearer JWT. Requires can_manage_or_own_events().
- **Request body**: event_id (required)
- **Success response**: 200 — {"message": "Event deleted successfully."}
- **Errors**: 400 missing event_id; 404 event not found; 400 "No estate found…"; 403 insufficient privileges; 500 if the delete fails.

GET /event/{event_id}

Retrieves details of a single event, including creator name.

- **Auth**: Bearer JWT. Requires can_manage_or_own_events().
- **Request**: path param event_id
- **Success response**: 200 — {"event": [{"id", "name", "max_num_of_attendees", "start_time", "end_time", "creator_id", "creator_name"}]}
- **Errors**: 400 missing event_id; 404 event not found; 400 "No estate found…"; 403 insufficient privileges.

GET /event/creator/{account_holder_id}

Lists events created by a specific account holder, paginated.

- **Auth**: Bearer JWT. Requires can_manage_or_own_events() for account_holder_id.
- **Request**: path param account_holder_id; query params page, page_size
- **Success response**: 200 — {"events": [...], "pagination": {...}}
- **Errors**: 400 missing account_holder_id; 400 invalid pagination params; 400 "No estate found…"; 403 insufficient privileges.

GET /event/estate/{estate_id}

Lists all events in an estate (joined across all account holders), paginated.

- **Auth**: Bearer JWT. Requires admin, or MANAGE_ALL/MANAGE_EVENTS privilege within estate_id (direct DB check).
- **Request**: path param estate_id; query params page, page_size
- **Success response**: 200 — {"events": [...], "pagination": {...}} (each event includes creator_id/creator_name)
- **Errors**: 400 missing estate_id; 400 invalid pagination params; 403 insufficient privileges.

Event Register API

Routes are prefixed /event_register/... and implemented in lambdas/event_register/app.py / lambdas/event_register/handlers/event_register.py. This module registers guests for an event — either created directly by an authenticated account holder, or self-service via a signed invite token — and manages the resulting event_registers link records.

POST /event_register/create

Creates a brand-new guest and registers them for an event in one step, generating an OTP scoped to the event's time window.

- **Auth**: Bearer JWT. Requires `can_manage_or_own_events()` on the event's creator.
- **Request body**: `account_holder_id` (required — the guest's creator/account holder), `event_id` (required), `name` (required), `id_type` (required), `id_num` (required), `contact` (required), `other_info` (optional, merged with generated `otp`/`purpose`/`entry_window`)
- **Success response**: `201` — `{"message": "...", "event_register_id": <int>, "guest_id": <int>, "personal_info_id": <int>}`
- **Errors**: `400` missing required fields; `404` event not found; `400` "No estate found…"; `403` insufficient privileges; `400` if `account_holder_id`'s estate can't be determined.

POST /event_register/invite/use

Public self-registration endpoint: consumes a signed invite token (from `/event/invite/generate`) to create a guest and register them for the event, without requiring the caller to be logged in.

- **Auth**: none (no `verify_token` call) — authorization is entirely via the signed `token` query parameter, which is decoded with the same `JWT_SECRET` and must not be expired/invalid.
- **Request**: query param `token` (required, invite JWT containing `event_id`, `account_holder_id`, `event_name`, `start_time`, `end_time`); body fields `name` (required), `id_type` (required), `id_num` (required), `contact` (required)
- **Success response**: `201` — `{"message": "...", "event_register_id": <int>, "guest_id": <int>, "personal_info_id": <int>, "otp": <str>}`
- **Errors**: `400` missing token; `403` token expired or invalid; `400` missing guest detail fields; `404` creator account not found.

GET /event_register/event/{event_id}

Lists guests registered for a given event, paginated.

- **Auth**: Bearer JWT. Requires `can_manage_or_own_events()` on the event's creator.
- **Request**: path param `event_id`; query params `page`, `page_size`
- **Success response**: `200` — `{"event_id": <str>, "guests": [{"guest_id", "guest_name", "contact", "estate_id"}, ...], "pagination": {...}}`
- **Errors**: `400` missing `event_id`; `404` event not found; `400` "No estate found…"; `403` insufficient privileges; `400` invalid pagination params.

DELETE /event_register/delete

Removes a guest from an event by deleting the guest's `personal_info` record, which cascades to remove the guest and their `event_registers` entry entirely (not just the event link).

- **Auth**: Bearer JWT. Requires `can_manage_or_own_events()` on the event's creator.
- **Request body**: `event_id` (required), `guest_id` (required)
- **Success response**: `200` — `{"message": "Guest removed from event successfully, all related data deleted."}`
- **Errors**: `400` missing `event_id`/`guest_id`; `404` event not found; `400` "No estate found…"; `403` insufficient privileges; `500` if the delete fails.

Face API

Handles biometric guest check-in/check-out via AWS Rekognition, and ad-hoc vehicle image uploads to S3. Implemented in `lambdas/face/app.py`, dispatching to `lambdas/face/handlers/pedestrian.py`. All three endpoints require a valid Bearer JWT; the save/match endpoints are additionally restricted to gate **TABLET** device tokens (tokens minted by the Device Token API's `generate_tablet_token`).

POST /face/save

Registers a guest's face for a `CHECKED_IN` visit: detects and indexes the face in the Rekognition collection `gate_app_collection_id`, uploads the image to the `itc-lpr-bucket` S3 bucket, records a `faces` row, flips the guest's status to `CHECKED_IN`, sends a push notification to the inviting user, and writes a `FACE_SAVED` log entry.

- **Auth**: Requires Bearer JWT (`verify_token`). The decoded token must carry a `gate_id` and `type == "TABLET"`, otherwise `401 Unauthorized`.
- **Request body**:
  - `inviter_user_id` (required) — id of the user field returned in the response payload path (not directly used beyond the required-fields check)
  - `guest_id` (required) — guest whose face is being registered
  - `guest_name` (required) — used for S3 object metadata and the push-notification text
  - `image` (required) — base64-encoded face image
- **Success response**: `201 {"message": "Face saved successfully."}`. Note: if Rekognition's `detect_faces` finds no `FaceDetails` in the image, the handler silently skips all DB/S3/notification work and still returns this same `201` success message.
- **Error responses**:
  - `401` — missing/invalid/expired token, or token is not a TABLET token with a `gate_id`
  - `400` — one of `inviter_user_id`, `guest_id`, `guest_name`, `image` missing; or the guest does not exist / `personal_info.expiry` has passed (`"Guest not found or access has expired."`); or the guest row could not be updated (`"Guest does not exist or could not be updated."`)
  - `500` — unhandled exception (e.g., DB error, `DATABASE_URL` not set)

POST /face/match

Looks up a submitted face image against the Rekognition collection, verifies the matched face belongs to a non-expired guest, marks the guest `CHECKED_OUT`, deletes the `faces` row and the Rekognition face record, notifies the inviting user, and writes a `FACE_MATCH` log entry.

- **Auth**: Requires Bearer JWT. Token must carry `gate_id` and `type == "TABLET"`, otherwise `401 Unauthorized`.
- **Request body**:
  - `image` (required) — base64-encoded face image to search with
- **Success response**: `200 {"message": "Match successful."}`
- **Error responses**:
  - `401` — missing/invalid/expired token, or not a TABLET token with `gate_id`
  - `404` — no Rekognition match corresponds to a valid (non-expired) face in the `faces` table (`"Face not found or guest access has expired."`), or the face row was already gone by delete time (`"Face not found in DB."`)
  - `400` — the guest linked to the face could not be updated (`"Guest does not exist or could not be updated."`)
  - `500` — unhandled exception (Rekognition or DB error)
  - Note: if Rekognition returns no `FaceMatches` at all, the function returns Python `None` rather than a proper API Gateway response dict — a latent bug that will surface as a Lambda invocation error rather than a clean HTTP error.

POST /face/upload

Uploads an arbitrary vehicle/LPR image directly to the `lpr-images-867721879638` S3 bucket under `lpr-images/{file_name}`, tagged with a `key` metadata value. Not tied to Rekognition or the `faces`/`guests` tables.

- **Auth**: Requires Bearer JWT (`verify_token`) — any valid token; no admin/privilege or device-type check.
- **Request body**:

  - `image` (required) — base64-encoded image bytes
  - `file_name` (required) — used to build the S3 object key
  - `key` (required) — stored as S3 object metadata (`{"key": ...}`)
- **Success response**: `200` `{"message": "Match successful."}` (message text is a carry-over from the face-match code path; this endpoint does not perform any matching)
- **Error responses**:
  - `401` — missing/invalid/expired token
  - `500` — S3 `put_object` failure (`"Internal Server Error."`), or any unhandled exception including a `KeyError` if `image`/`file_name`/`key` are absent from the body (these fields are read with `body["..."]`, not `.get()`, so a missing field raises and is caught by the outer handler as a generic `500`)

Device Token API

Issues short-lived, purpose-specific JWTs for unattended devices (gate controllers and tablets) that then authenticate to other endpoints (e.g. Face API) using `gate_id`/`type` claims instead of a user session. Implemented in `lambdas/device_token/app.py`, dispatching to `lambdas/device_token/handlers/device_token.py`.

POST /device_token/generate_controller_token

Mints a JWT for a physical gate controller device, scoped to one `gate_id`.

- **Auth**: Requires Bearer JWT (user session). Caller must be a global admin (`is_admin: true` in the token) OR hold `MANAGE_ALL`/`MANAGE_DEVICES` privilege for the estate that owns the gate, resolved via `get_user_privileges_for_gate` → `PrivilegeContext.can_manage_devices()`.
- **Request body**:
  - `gate_id` (required) — the gate the controller token will be scoped to
- **Success response**: `200` `{"gate_controller_token": "<jwt>"}`. The JWT payload is `{"gate_id": ..., "type": "GATE"}`, signed HS256 with `JWT_SECRET`.
- **Error responses**:
  - `400` — missing `gate_id`
  - `403` — caller has no account holder in the gate's estate (`"You do not have an account in the estate for this gate"`), or lacks `MANAGE_ALL`/`MANAGE_DEVICES` (`"Not authorized to generate a gate controller token for this estate."`)
  - (from `verify_token`) `401` — missing/invalid/expired token
  - `500` — unhandled exception
- Note: the code that looks up the gate's estate/name and writes a `GATE_CONTROLLER_TOKEN_GENERATION` log entry sits under the `if not privileges.can_manage_devices(): return ...` block after the `return` statement, so due to indentation it is unreachable dead code — successful controller-token generation does **not** actually write a log entry despite the docstring's intent.

POST /device_token/generate_tablet_token

Mints a JWT for a gate tablet device, scoped to one `gate_id`.

- **Auth**: Same as above — Bearer JWT required; caller must be a global admin or hold `MANAGE_ALL`/`MANAGE_DEVICES` for the gate's estate via `get_user_privileges_for_gate` → `can_manage_devices()`.
- **Request body**:
  - `gate_id` (required) — the gate the tablet token will be scoped to
- **Success response**: `200` `{"tablet_token": "<jwt>"}`. JWT payload is `{"gate_id": ..., "type": "TABLET"}`, signed HS256. (This is the token type the Face API's `/face/save` and `/face/match` endpoints require.)
- **Error responses**:
  - `400` — missing `gate_id`

  - 403 — no account holder in the gate's estate ("You do not have an account in the estate for this gate"), or lacks MANAGE_ALL/MANAGE_DEVICES ("Not authorized to generate a tablet token for this estate.")
  - 401 — missing/invalid/expired token
  - 500 — unhandled exception
- On success this handler correctly logs a TABLET_TOKEN_GENERATION entry and commits before returning the token.

Log API

Read-only endpoints for retrieving and searching audit-log entries (the logs table), scoped either by estate or by account holder. Implemented in lambdas/log/app.py, dispatching to lambdas/log/handlers/log.py. All four endpoints are GET and share the same privilege model: caller must be a global admin, or have MANAGE_ALL/VIEW_LOGS privilege in the relevant estate (checked via a raw SQL EXISTS query joining account_holders → personal_info → account_types → account_type_privilege_links → privileges, not the PrivilegeContext helper).

GET /log/estate/{estate_id}

Returns a paginated list of logs for an estate within a date range, excluding GATE_CONTROLLER_HEARTBEAT entries.

- **Auth**: Requires Bearer JWT. Caller must be admin or hold MANAGE_ALL/VIEW_LOGS for the estate.
- **Path param**: estate_id (required)
- **Query params**:
  - start_date, end_date (required) — ISO-8601 timestamps (accepts trailing Z)
  - page (optional, default 1), page_size (optional, default 20, max 100)
- **Success response**: 200 {"estate_id": ..., "logs": [...], "pagination": {"page", "page_size", "total", "pages"}}
- **Error responses**:
  - 400 — missing estate_id; missing start_date/end_date; invalid ISO-8601 dates; invalid page/page_size
  - 403 — caller lacks admin/MANAGE_ALL/VIEW_LOGS for the estate
  - 401 — missing/invalid/expired token
  - 500 — unhandled exception

GET /log/account_holder/{account_holder_id}

Returns a paginated list of logs where the given account holder is either the target (target_account_holder_id) or the actor (via the account holder's user_id matching actor_user_id).

- **Auth**: Requires Bearer JWT. Caller must be admin, hold MANAGE_ALL/VIEW_LOGS in the account holder's estate, or **be** that account holder (i.e., account_holders.user_id = caller's user_id AND id = account_holder_id).
- **Path param**: account_holder_id (required)
- **Query params**:
  - start_date, end_date — read and parsed with datetime.fromisoformat unconditionally before the account_holder_id presence check; if either is missing or malformed this raises an unhandled exception, which the outer app.py except block converts to a generic 500 (unlike /log/estate/{estate_id}, there is no explicit 400 guard here)
  - page (optional, default 1), page_size (optional, default 20, max 100)
- **Success response**: 200 {"account_holder_id": ..., "logs": [...], "pagination": {"page", "page_size", "total", "pages"}}
- **Error responses**:
  - 400 — missing account_holder_id; invalid page/page_size

  - 403 — caller is neither privileged (MANAGE_ALL/VIEW_LOGS) in that estate nor the account holder themselves ("You do not have privileges to view logs for this account holder.")
  - 401 — missing/invalid/expired token
  - 500 — unhandled exception (including missing/invalid start_date/end_date, see above)

GET /log/search_account_holder/{account_holder_id}

Like /log/account_holder/{account_holder_id} but supports free-text filtering, and treats the date range as optional.

- **Auth**: Same as /log/account_holder/{account_holder_id} — admin, MANAGE_ALL/VIEW_LOGS in the account holder's estate, or the account holder themselves.
- **Path param**: account_holder_id (required)
- **Query params** (all optional):
  - plate — substring matched (ILIKE %value%) against the log message
  - name — substring matched (ILIKE %value%) against the log message
  - start_date, end_date — ISO-8601; if both given, filters date_created BETWEEN; malformed values return 400
  - page (default 1), page_size (default 20) — no explicit min/max validation beyond int() parsing succeeding
- **Success response**: 200 {"account_holder_id": ..., "logs": [...], "pagination": {"page", "page_size", "total", "pages"}}
- **Error responses**:
  - 400 — missing account_holder_id; non-integer page/page_size; invalid date format
  - 403 — caller lacks privilege and is not the account holder ("Access denied.")
  - 401 — missing/invalid/expired token
  - 500 — unhandled exception

GET /log/search_estate/{estate_id}

Like /log/estate/{estate_id} but supports free-text filtering, and treats the date range as optional (dates and heartbeat exclusion both apply as filters).

- **Auth**: Requires Bearer JWT. Caller must be admin or hold MANAGE_ALL/VIEW_LOGS for the estate.
- **Path param**: estate_id (required)
- **Query params** (all optional):
  - plate — substring matched (ILIKE %value%) against the log message
  - name — substring matched (ILIKE %value%) against the log message
  - start_date, end_date — ISO-8601; if both given, filters date_created BETWEEN; malformed values return 400
  - page (default 1), page_size (default 20) — no explicit min/max validation beyond int() parsing succeeding
- Results always exclude type = 'GATE_CONTROLLER_HEARTBEAT' logs.
- **Success response**: 200 {"estate_id": ..., "logs": [...], "pagination": {"page", "total", "pages"}} (note: unlike the other three log endpoints, this response's pagination object omits page_size)
- **Error responses**:
  - 400 — missing estate_id; non-integer page/page_size; invalid date format
  - 403 — caller lacks admin/MANAGE_ALL/VIEW_LOGS for the estate ("Access denied.")
  - 401 — missing/invalid/expired token
  - 500 — unhandled exception

Account Type Privilege Link API

Manages the many-to-many mapping between account_types and privileges (the account_type_privilege_links join table), which drives what each account type is permitted to do within an estate. Implemented in

lambdas/account_type_privilege_link/app.py, dispatching to lambdas/account_type_privilege_link/handlers/acc
All three endpoints require the caller's JWT to have is_admin: true — this is a global-admin-only API with no estate-scoped privilege fallback.

POST /account_type_privilege_link/create

Assigns a privilege to an account type, after confirming both referenced rows exist.

- **Auth**: Requires Bearer JWT with is_admin: true; otherwise 403.
- **Request body**:
  - account_type_id (required)
  - privilege_id (required)
- **Success response**: 201 {"message": "Privilege assigned successfully."}. Also writes an ACCOUNT_TYPE_PRIVILEGE_LINK_CREATION log entry.
- **Error responses**:
  - 400 — missing account_type_id or privilege_id
  - 403 — caller is not admin ("Access denied. Only admins can assign privileges.")
  - 404 — account_type_id not found ("Account type not found."), or privilege_id not found ("Privilege not found.")
  - 409 — the link already exists (insert uses ON CONFLICT (account_type_id, privilege_id) DO NOTHING, and a no-op insert is treated as a conflict: "Privilege already assigned to this account type.")
  - 401 — missing/invalid/expired token
  - 500 — unhandled exception

DELETE /account_type_privilege_link/remove

Removes an existing account-type/privilege link.

- **Auth**: Requires Bearer JWT with is_admin: true; otherwise 403.
- **Request body**:
  - account_type_id (required)
  - privilege_id (required)
- **Success response**: 200 {"message": "Privilege removed successfully."}. Also writes an ACCOUNT_TYPE_PRIVILEGE_LINK_REMOVAL log entry.
- **Error responses**:
  - 400 — missing account_type_id or privilege_id
  - 403 — caller is not admin ("Access denied. Only admins can remove privileges.")
  - 404 — no matching link existed to delete ("Privilege link not found.")
  - 401 — missing/invalid/expired token
  - 500 — unhandled exception

GET /account_type_privilege_link/account_type/{account_type_id}

Lists the privileges (id + name) currently assigned to an account type, paginated.

- **Auth**: Requires Bearer JWT with is_admin: true; otherwise 403.
- **Path param**: account_type_id (required)
- **Query params**: page (optional, default 1), page_size (optional, default 20, max 100)
- **Success response**: 200 {"privileges": [{"id", "name"}, ...], "pagination": {"page", "page_size", "total", "pages"}}, ordered by privilege name.
- **Error responses**:
  - 400 — missing account_type_id; invalid page/page_size
  - 403 — caller is not admin ("Access denied. Only admins can view privileges.")
  - 401 — missing/invalid/expired token
  - 500 — unhandled exception

Gate Access Group Link API

Manages the many-to-many mapping between `gates` and `access_groups` (the `gate_access_group_links` table), including per-link optional `opening_time`/`closing_time` overrides. Implemented in `lambdas/gate_access_group_li` dispatching to `lambdas/gate_access_group_link/handlers/gate_access_group_link.py`. All endpoints require Bearer JWT; write endpoints and the "by gate" list additionally require `MANAGE_ALL`/`MANAGE_ACCESS` privilege (`PrivilegeContext.can_manage_access_groups()`) resolved for the gate's or access group's estate.

POST /gate_access_group_link/create

Links an access group to a gate, optionally with custom `opening_time`/`closing_time`.

- **Auth**: Requires Bearer JWT. Caller must be admin or hold `MANAGE_ALL`/`MANAGE_ACCESS` for the estate that owns the gate, resolved via `get_user_privileges_for_gate`.
- **Request body**:
  - `gate_id` (required)
  - `access_group_id` (required)
  - `opening_time` (optional) — if omitted, DB column default applies
  - `closing_time` (optional) — if omitted, DB column default applies
- **Success response**: `201 {"message": "Access group linked successfully."}`. Writes an `ACCESS_GROUP_GATE_LINK_CREATION` log entry.
- **Error responses**:
  - `400` — missing `gate_id`/`access_group_id`; or gate and access group belong to different estates (`"Estate mismatch: gate and access group belong to different estates."`)
  - `404` — gate not found, or access group not found
  - `403` — caller has no account holder in the gate's estate, or lacks `MANAGE_ALL`/`MANAGE_ACCESS` (`"You do not have privileges to manage access group links."`)
  - `409` — link already exists (`ON CONFLICT ... DO NOTHING` treated as conflict: `"Access group already linked to this gate."`)
  - `401` — missing/invalid/expired token
  - `500` — unhandled exception

POST /gate_access_group_link/update

Updates `opening_time`/`closing_time` on an existing gate/access-group link; omitted fields retain their current value (`COALESCE`).

- **Auth**: Requires Bearer JWT. Caller must be admin or hold `MANAGE_ALL`/`MANAGE_ACCESS` for the gate's estate.
- **Request body**:
  - `gate_id` (required)
  - `access_group_id` (required)
  - `opening_time` (optional)
  - `closing_time` (optional)
- **Success response**: `200 {"message": "Access group link updated successfully."}`. Writes an `ACCESS_GROUP_GATE_LINK_UPDATE` log entry.
- **Error responses**:
  - `400` — missing `gate_id`/`access_group_id`; estate mismatch between gate and access group
  - `404` — no existing link between the given gate and access group
  - `403` — no account holder in the gate's estate, or lacks `MANAGE_ALL`/`MANAGE_ACCESS` (`"You do not have privileges to update access group links."`)
  - `401` — missing/invalid/expired token
  - `500` — unhandled exception

DELETE /gate_access_group_link/remove

Removes a gate/access-group link.

- **Auth**: Requires Bearer JWT. Caller must be admin or hold `MANAGE_ALL`/`MANAGE_ACCESS` for the gate's estate (checked via `get_user_privileges_for_gate` directly, without first validating that the gate and access group belong to the same estate — this endpoint does not perform the estate-mismatch check present in create/update).
- **Request body**:
  - `gate_id` (required)
  - `access_group_id` (required)
- **Success response**: `200 {"message": "Access group removed successfully."}`. Writes an `ACCESS_GROUP_GATE_LINK_REMOVAL` log entry.
- **Error responses**:
  - `400` — missing `gate_id`/`access_group_id`
  - `403` — no account holder in the gate's estate, or lacks `MANAGE_ALL`/`MANAGE_ACCESS` (`"You do not have privileges to manage access group links"`)
  - `404` — no matching link found to delete
  - `401` — missing/invalid/expired token
  - `500` — unhandled exception

GET /gate_access_group_link/gate/{gate_id}

Lists access groups (id + name) linked to a given gate, paginated.

- **Auth**: Requires Bearer JWT. Caller must be admin or hold `MANAGE_ALL`/`MANAGE_ACCESS` for the gate's estate (`get_user_privileges_for_gate`).
- **Path param**: `gate_id` (required)
- **Query params**: `page` (optional, default `1`), `page_size` (optional, default `20`, max `100`)
- **Success response**: `200 {"access_groups": [{"access_group_id", "access_group_name"}, ...], "pagination": {"page", "page_size", "total", "pages"}}`, ordered by access group name.
- **Error responses**:
  - `400` — missing `gate_id`; invalid `page`/`page_size`
  - `403` — no account holder in the gate's estate (`"You do not have an account in the estate for this gate"`), or lacks `MANAGE_ALL`/`MANAGE_ACCESS` (`"You do not have privileges to retrieve access groups related to this gate"`)
  - `401` — missing/invalid/expired token
  - `500` — unhandled exception

GET /gate_access_group_link/access_group/{access_group_id}

Lists gates (id + name) linked to a given access group, paginated.

- **Auth**: Requires Bearer JWT. Caller must be admin or hold `MANAGE_ALL`/`MANAGE_ACCESS` for the access group's estate, resolved via `get_user_privileges_for_access_group` (looks up the access group's `estate_id`, then delegates to the estate-level privilege check).
- **Path param**: `access_group_id` (required)
- **Query params**: `page` (optional, default `1`), `page_size` (optional, default `20`, max `100`)
- **Success response**: `200 {"gates": [{"gate_id", "gate_name"}, ...], "pagination": {"page", "page_size", "total", "pages"}}`, ordered by gate name.
- **Error responses**:
  - `400` — missing `access_group_id`; invalid `page`/`page_size`
  - `403` — access group not found or caller has no account holder in that estate (`"Access group not found or you do not have an account in this estate"`), or lacks `MANAGE_ALL`/`MANAGE_ACCESS` (`"You do not have privileges to retrieve gates for this access group"`)
  - `401` — missing/invalid/expired token

– `500` — unhandled exception